AGENTIC AI FRAMEWORK FOR THREAT DETECTION IN WEB BASED CHAT PLATFORMS

A PROJECT REPORT

Submitted by

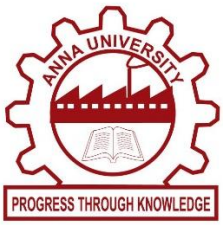
MANJUSHREE R G

SYNDHAVI S

DIVIYADHARSHINI R S

COURSE CODE: CS23604

COURSE TITLE: CREATIVE AND INNOVATIVE PROJECT



Department of Computer Technology

MIT CAMPUS, ANNA UNIVERSITY

CHENNAI 600 025

APRIL 2026

DEPARTMENT 0F COMPUTER TECHNOLOGY
ANNA UNIVERSITY, MIT CAMPUS
CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report **"AGENTIC AI FRAMEWORK FOR THREAT DETECTION IN WEB BASED CHAT PLATFORMS"** is the bonafide work of **"MANJUSHREE R G (2023503524), SYNDHAVI S (2023503548), DIVIYADHARSHINI R S (2023503527)"** who carried out the project work under my supervision.

SIGNATURE

DR. P. PABITHA
SUPERVISOR
Associate Professor
Department of Computer Technology
Anna University, MIT Campus
Chennai 600 044

SIGNATURE

DR. PONSY RK SATHIA BHAMA
HEAD OF DEPARTMENT
Associate Professor
Department of Computer Technology
Anna University, MIT Campus
Chennai 600 044

ABSTRACT

The widespread adoption of online chat platforms by minors has led to an alarming rise in cyber threats including online grooming, cyberbullying, and exposure to harmful content. Conventional content moderation systems operate in a cloud-based, reactive paradigm that processes harmful messages only after they have already been delivered, introducing unacceptable delays and raising serious privacy concerns when sensitive chat data is transmitted to remote servers.

This project proposes an Agentic AI Framework for Threat Detection in Web-Based Chat Platforms a privacy-preserving, edge-based system that analyzes chat messages in real time directly on the user's device using a Chrome extension as the interception layer. The framework integrates three specialized intelligent modules: a Threat Detection Module powered by a fine-tuned DistilBERT model combined with engineered numerical features such as profanity score, sentiment score, capital ratio, message length, and repetition score; a Context Risk Assessment Module that maintains an episodic memory window and computes a weighted contextual risk score by analyzing historical threat patterns, frequency of harmful messages, and escalation trends; and an Intrusion Safety Agent driven by a quantized Phi-2 language model that decides upon appropriate safety actions.

The system performs three escalating safety actions: WARN (display warning to user), BLOCK (temporarily block the conversation), and ALERT_GUARDIAN (notify the guardian via email with an unlock link). The backend is implemented as a Flask API and the storage layer uses

IndexedDB along with Chrome's local storage for persisting guardian configuration and thresholds.

The system was trained on a balanced dataset of approximately 79,994 samples derived from six publicly available cyberbullying and toxicity datasets. The fine-tuned DistilBERT hybrid model achieved an accuracy of 86.6%, precision of 88.2%, recall of 84.5%, and an F1 score of 86.3%. The proposed framework demonstrates that edge-based agentic AI can provide timely, proactive, and privacy-conscious protection for minors in online communication environments.

ACKNOWLEDGEMENT

We take this humble opportunity to thank **Dr. P. Jayashree**, Dean, MIT Campus, Anna University, and **Dr. Ponsy R K Sathia Bhama**, Associate Professor & Head, Department of Computer Technology, MIT Campus, Anna University for providing all the lab facilities in pursuit of this project.

Undertaking this project has helped us learn a lot, and we would like to express our sincere gratitude towards our Project Guide **Dr. P. Pabitha**, Associate Professor, Department of Computer Technology, MIT, Anna University, whose kind guidance, and proper directions helped us in shaping this project perfectly.

We acknowledge the feedback of panel members **Dr. Ponsy R K Sathia Bhama**, Associate Professor, Department of Computer Technology, MIT, Anna University, **Dr. T. Sudhakar**, Assistant Professor, Department of Computer Technology, MIT, Anna University, in reviewing our work, and encouraging us to successfully implement our project.

We thank all the teaching and non-teaching members of the Department of Computer Technology, MIT, Anna University, seniors, juniors, and friends whose appreciable help, either directly or indirectly helped in the smooth completion of the Creative and Innovative Project in a limited time frame.

Manjushree R G [2023503524]
Syndhavi S [2023503548]
Diviyadharshini R S [2023503527]

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iii**
	LIST OF TABLES	**viii**
	LIST OF FIGURES	**ix**
	LIST OF SYMBOLS AND ABBREVIATIONS	**x**
1.	**INTRODUCTION**	**1**
	1.1 Background and Motivation	1
	1.2 Problem Statement	3
	1.3 Proposed Solution	4
	1.4 Objectives	5
	1.5 Scope of the Project	5
	1.6 Organization of the Report	6
	1.7 Summary	6
2.	**LITERATURE REVIEW**	**8**
	2.1 General	8
	2.2 Review of Related Works	8
	2.3 Research Gaps Identified	11
	2.4 Summary	13
3.	**SYSTEM DESIGN AND ARCHITECTURE**	**14**
	3.1 System Overview	14
	3.2 Architecture Diagram	14
	3.3 Chrome Extension Layer	15
	3.4 AI Safety Engine Backend	16
	3.5 Storage Layer	18

	3.6 Algorithm for the Proposed System	18
	3.7 Summary	22
4.	**IMPLEMENTATION**	**24**
	4.1 Dataset Overview and Preprocessing	24
	4.2 Threat Detection Module	26
	4.3 Context Risk Assessment Module	26
	4.4 Intervention Agent Module	27
	4.5 Flask API and Backend Integration	27
	4.6 Summary	28
5.	**RESULTS AND DISCUSSION**	29
	5.1 Evaluation Metrics	29
	5.2 Experimental Results	30
	5.3 System Output Demonstration	31
	5.4 Discussion	36
	5.5 Summary	37
6.	**CONCLUSION AND FUTURE WORK**	39
	6.1 Conclusion	39
	6.2 Future Work	40
	REFERENCES	**42**

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Literature Review Summary	11
4.1	Dataset Overview	24
4.2	Feature Engineering Summary	25
5.1	Evaluation Metrics and Formulas	29
5.2	Model Performance Results	30
5.3	Comparison with Baseline Models	30

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
3.1	AI Safety Engine Architecture	15
5.1	System Output - Without LLM	32
5.2	System Output - With LLM (Phi-2)	32
5.3	System Output - With Chrome Extension (Allow)	32
5.4	System Output - With Chrome Extension (Warn)	33
5.5	System Output - With Chrome Extension (Block)	34
5.6	System Output - With Chrome Extension (Alert)	35

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

ABBREVIATION	DESCRIPTION
AI	Artificial Intelligence
LLM	Large Language Model
NLP	Natural Language Processing
DistilBERT	Distilled Bidirectional Encoder Representations from Transformers
Phi-2	Microsoft Phi-2 Quantized Language Model
API	Application Programming Interface
DOM	Document Object Model
SMTP	Simple Mail Transfer Protocol
CORS	Cross-Origin Resource Sharing
TP	True Positive
TN	True Negative
FP	False Positive
FN	False Negative
TDA	Threat Detection Accuracy
FPR	False Positive Rate
F1	F1 Score (Harmonic Mean of Precision and Recall)
sk	Threat Score of message k
rk	Contextual Risk Score of message k
Hk	Historical Context Window
$\tau 1$, $\tau 2$	Risk Level Thresholds
W	Conversation Window Size
ERP	Enterprise Resource Planning
MCP	Multi-step Control Pipeline
UUID	Universally Unique Identifier

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

The proliferation of digital communication platforms has fundamentally transformed the way people interact with one another. Online chat platforms, social media networks, and instant messaging services have become an integral part of daily life, particularly for younger users. According to research published by the Pew Research Center, a significant majority of teenagers and pre-adults are active users of platforms such as Instagram, TikTok, Facebook, and various messaging applications. While these platforms offer enormous benefits in terms of connectivity and information sharing, they also expose minors to serious risks including cyberbullying, online grooming, hate speech, and suicidal content.

Online grooming, in particular, is a severe and growing concern. It refers to the process by which an adult systematically builds trust with a minor over time through digital conversations with the intent to exploit or abuse them. Unlike direct cyberbullying, which often manifests in single hostile messages, grooming occurs gradually and progressively making it particularly difficult for conventional reactive monitoring systems to detect. Similarly, cyberbullying involves repeated targeted harassment that may not trigger alert thresholds when individual messages are evaluated in isolation.

Current content moderation systems predominantly operate in a cloud-based, centralized fashion. These systems intercept user messages, transmit them to remote servers for analysis, and return moderation decisions. This architecture introduces multiple significant drawbacks: it creates latency in the moderation response, meaning harmful messages may already have been read by the target before any action is taken; it raises serious privacy concerns since sensitive and potentially personally identifiable conversation data is transmitted over networks to third-party servers; and it provides no mechanism for proactive early-stage intervention before a situation escalates.

Agentic AI represents a new generation of artificial intelligence systems designed to operate autonomously as intelligent agents that perceive their environment, maintain memory of past interactions, plan multi-step actions, and make context-aware decisions to achieve specific goals. Unlike traditional AI models that merely respond to individual inputs, agentic systems can coordinate multiple specialized modules in a pipeline to perform complex real-world tasks with minimal human intervention. This paradigm offers a powerful foundation for building a real-time, context-aware, and proactive chat safety system.

Motivated by these considerations, this project proposes an Agentic AI Framework for Threat Detection in Web-Based Chat Platforms. The system is deployed as a Chrome browser extension combined with an edge-based backend AI Safety Engine, enabling real-time analysis of chat messages directly on the user's device without requiring sensitive data to be transmitted to external servers.

1.2 Problem Statement

The use of online chat platforms by minors has increased dramatically, bringing with it a corresponding rise in risks such as online grooming, cyberbullying, hate speech, and exposure to suicidal content. Most existing content monitoring systems operate in a cloud-based and reactive manner, meaning they identify harmful messages only after they have already been delivered and read. This approach introduces two critical failures:

- Delayed Response: By the time a harmful message is flagged and moderated, the psychological damage to the minor may have already occurred. This is especially critical in grooming scenarios where a single apparently benign message may be part of a longer pattern of manipulation.
- Privacy Violations: Transmitting sensitive chat conversations to remote servers for analysis exposes users to risks of data breaches and unauthorized surveillance. For minors, this is particularly unacceptable from both ethical and legal standpoints.

Furthermore, existing systems predominantly rely on monolithic machine learning classifiers that evaluate messages in isolation without considering conversational history. This makes them inherently incapable of detecting progressive, context-dependent threats such as grooming, which unfolds gradually over multiple messages and may not be recognizable from any single message alone.

There is thus a clear and urgent need for a system that can: analyse chat messages in real time on the user's device to eliminate cloud-based privacy risks; consider the full conversational context and history when assessing

threat levels; distinguish between levels of threat severity and take graduated, proportionate actions; and intervene proactively before situations escalate to serious harm.

1.3 Proposed Solution

This project proposes an edge-based Agentic AI Framework called the AISafetyEngine that addresses all identified problems. The system is deployed through a Chrome extension that intercepts chat messages using DOM manipulation and content scripting. The intercepted messages are sent to a locally running Flask API backend that houses three specialized AI agents:

- Threat Detection Module: A fine-tuned DistilBERT model combined with engineered numerical features (profanity score, sentiment score, capital ratio, message length, repetition score) that assigns a threat score between 0 and 1 to each incoming message.
- Context Risk Assessment Module: A stateful agent that maintains a sliding window of recent threat scores and computes a weighted contextual risk score by analysing historical threat patterns, frequency of harmful messages, and an escalation factor that increases when threats persist.
- Intrusion Safety Agent: A quantized Phi-2 language model that receives the message, threat score, risk score, and conversation history, and decides upon the final safety action from ALLOW, WARN, BLOCK, or ALERT_GUARDIAN.

The system communicates results back to the Chrome extension, which visually displays warnings, blocks message display, or triggers guardian email alerts through the SMTP API. This architecture ensures sub-second response times and complete on-device data privacy.

1.4 Objectives

The key objectives of this project are:

- To design a privacy-preserving, edge-based AI system for monitoring online conversations involving minors.
- To develop a threat detection module using fine-tuned DistilBERT that identifies potentially harmful messages at the message level with high accuracy.
- To implement a contextual risk assessment module that analyzes conversation history, interaction patterns, and escalation trends to compute a holistic risk score.
- To design an intelligent intervention module powered by a quantized Phi-2 LLM that decides appropriate actions such as warnings, message blocking, or guardian alerts based on risk severity.
- To validate the effectiveness of the proposed system using publicly available and synthetic datasets through a prototype implementation deployed as a Chrome browser extension.

1.5 Scope of the Project

The scope of this project encompasses the following:

- Development and fine-tuning of a DistilBERT-based hybrid classification model for harmful message detection.
- Implementation of a context-aware risk scoring engine that maintains episodic memory across a conversation window.
- Development of an LLM-powered intervention decision agent using quantized Phi-2.

- Implementation of a Chrome browser extension for real-time DOM-based message interception.
- Development of a Flask REST API backend serving as the AI Safety Engine.
- Integration of a guardian notification system via SMTP email with token-based unlock functionality.
- Evaluation of the system on cyberbullying, toxicity, and hate speech datasets.

The system is scoped for web-based chat platforms accessible through the Google Chrome browser. Native mobile applications and end-to-end encrypted messaging platforms are outside the scope of this project.

1.6 Organization of the Report

The remainder of this report is organized as follows. Chapter 2 presents a comprehensive review of related literature in the areas of cyberbullying detection, agentic AI systems, edge intelligence, and online child safety. Chapter 3 describes the overall system design and architecture of the proposed AISafetyEngine framework. Chapter 4 details the implementation of each component including dataset preprocessing, model training, context agent, intervention agent, and backend API. Chapter 5 presents the results and performance evaluation of the system. Chapter 6 concludes the report and outlines directions for future work.

1.7 Summary

This chapter has established the motivation and problem context for the proposed AISafetyEngine. The rapid growth of online chat platform usage among minors, combined with the increasing sophistication of threats such as

online grooming and cyberbullying, underscores the urgent need for a proactive, privacy-preserving monitoring system. Existing cloud-based content moderation systems fail on two critical fronts: they introduce unacceptable response latency, and they expose sensitive conversation data to remote servers. The proposed solution addresses these limitations through an edge-based agentic AI framework that processes messages locally, in real time, directly on the user's device via a Chrome browser extension. The framework integrates three specialized intelligent agents a DistilBERT-based threat detection module, a context risk assessment agent with episodic memory, and a Phi-2 powered intervention agent to achieve context-aware, graduated, and proactive safety responses. The objectives, scope, and organization of the remainder of this report have been defined to systematically present the design, implementation, and evaluation of this system.

CHAPTER 2

LITERATURE REVIEW

2.1 General

The problem of detecting harmful content in online communications has been an active area of research for over a decade. The proliferation of social media platforms and chat applications has driven researchers to explore machine learning, natural language processing, and more recently, large language model-based approaches for automated content moderation. This chapter reviews the key works in this area, with a specific focus on cyberbullying detection, agentic AI frameworks, edge-based intelligence systems, and online child safety mechanisms.

2.2 Review of Related Works

[2]Dinakar et al. (2012) proposed one of the early computational approaches to cyberbullying detection using common sense reasoning. Their work, published in ACM Transactions on Interactive Intelligent Systems, focused on identifying and mitigating cyberbullying by leveraging common sense knowledge bases to understand the social context of messages. While their approach was pioneering, it was limited by the computational cost of real-time common sense inference and did not address the progressive nature of grooming threats.

[3]Ejaz, Razi, and Choudhury presented a comprehensive dataset for cyberbullying detection that incorporated not only aggressive texts but also features such as repetition, peer relationships, and intent to harm. This dataset-

centric contribution helped advance research by providing more nuanced labels for harmful content, moving beyond simple binary classification. However, their work was primarily focused on dataset construction and did not propose an end-to-end real-time detection system.

[4]Luo et al. (2025), in their paper published in IEEE Transactions on Cognitive Communications and Networking, introduced a multi-LLM edge intelligence architecture where lightweight language models collaborate under an orchestration layer to achieve low-latency and resource-efficient inference. Their Multi-LLM framework demonstrated the viability of deploying multiple language models at the network edge for intelligent applications. However, the work focused on general-purpose edge intelligence and did not address application-specific safety tasks such as contextual threat detection or the protection of minors in communication platforms.

[6]Mujtaba, Khowaja, and Dev (2025) proposed EdgeAIGuard, published in the IEEE Internet of Things Journal, which is one of the most closely related works to the proposed system. EdgeAIGuard proposes a multi-agent edge-based framework using LLM-powered Sentinel, Context, and Intervention agents to detect harmful content, analyse behavioural patterns, and protect minors in digital communication platforms. While this work validates the multi-agent agentic AI approach for child safety, it relies heavily on multiple LLM agents, increasing computational overhead. Furthermore, it does not explore lightweight client-side deployments such as browser extensions, which is the approach adopted in the proposed system.

[1]Acharya, Kuppan, and Divya (2025), in their survey published in IEEE Access, presented a comprehensive high-level overview of agentic AI systems. They characterized agentic AI as systems possessing coordinated

perception, reasoning, planning, memory, and decision-making modules capable of achieving complex goals autonomously. While their survey provides an excellent theoretical foundation for understanding agentic architectures, it is conceptual in nature and does not validate agentic AI in real-time safety-critical applications or in resource-constrained edge environments such as browser extensions.

[7]Sarferaz (2025) proposed an agentic AI framework integrated into Enterprise Resource Planning (ERP) systems to automate enterprise decision-making using autonomous agents and structured organizational data, published in IEEE Access. While demonstrating the applicability of agentic frameworks in complex organizational settings, this work is designed for structured enterprise environments and does not support unstructured conversational data, real-time intervention, or privacy-sensitive child safety applications.

Two papers published in IEEE Transactions on Software Engineering in 2026 addressed agentic systems in different contexts. The first presented a systematic evaluation of agentic AI systems using a frictionless parking scenario, providing architectural insights into how autonomous agents can be designed and evaluated. The second conducted a systematic literature review of agentic recommender systems, analyzing how autonomous agents incorporate memory, user modeling, and decision-making to generate personalized recommendations. Both works, while valuable for understanding agentic system design patterns, are conceptual and do not provide empirical evaluation in real-time, safety-critical deployment contexts.

The literature also includes significant work from [9]Thakkar et al. (2024) on AgentMerge, which explored enhancing generalization in fine-tuned LLM

agents through model merging techniques. This work provides insights into how specialized fine-tuned agents can be combined to improve overall agent performance, which is relevant to the multi-agent approach adopted in the proposed system.

Table 2.1: Literature Review Summary

S.No	Paper Title	Journal & Year	Proposed System	Limitations
1	Toward Edge General Intelligence With Multi-LLM	IEEE Trans. Cognitive Commun., Dec. 2025	Multi-LLM edge architecture for low-latency inference	No child safety or threat detection focus
2	EdgeAIGuard: Agentic LLMs for Minor Protection	IEEE IoT Journal, Sept. 2025	Multi-agent edge framework with Sentinel, Context, and Intervention agents	High compute overhead; no browser extension
3	Agentic AI: Comprehensive Survey	IEEE Access, 2025	Survey of agentic AI perception, memory, and planning modules	Conceptual; no real-time validation
4	Implementing Agentic AI Into ERP Software	IEEE Access, 2025	Agentic AI in enterprise ERP decision-making	No support for conversational or child safety data
5	Agentic AI Systems: Architecture and Evaluation	IEEE Trans. Software Eng., 2026	Evaluation using parking scenario	Conceptual; no empirical comparison
6	Agentic Recommender Systems: Literature Review	IEEE Trans. Software Eng., 2026	Survey of memory-based agentic recommender systems	No empirical evaluation

2.3 Research Gaps Identified

Based on the review of related literature, the following critical research gaps have been identified that motivate the proposed system:

Gap 1: Limited Context-Aware Threat Detection

Most existing online safety systems evaluate messages in isolation without considering conversational history. This approach fundamentally fails to detect progressive threats such as grooming, which develop gradually over

time through a series of individually innocuous-seeming messages. There is a clear need for a system that maintains episodic memory across a conversation and assesses risk based on sequential patterns.

Gap 2: High Latency and Privacy Concerns in Cloud-Based Moderation

Centralized cloud-based moderation systems require user conversation data to be transmitted to remote servers, introducing response latency and raising serious privacy concerns — especially when the users are minors. An edge-based approach that processes messages locally on the user's device is needed to address both latency and privacy simultaneously.

Gap 3: Lack of Agent-Based Modular Decision Making

Existing systems predominantly rely on monolithic machine learning models that lack modular reasoning and decision-making capabilities. The absence of a structured agentic architecture means these systems cannot adapt their responses based on multi-factor contextual assessments. A modular agentic design separating detection, contextual reasoning, and intervention is needed.

Gap 4: Limited Focus on Proactive Intervention Strategies

Current systems typically act only after severe harm has already occurred, focusing on content removal rather than early prevention. There is a need for an intervention mechanism capable of performing graduated, proactive actions — from warnings and message blocking to guardian alerts — based on assessed threat severity before harm escalates.

2.4 Summary

The literature review reveals that while significant progress has been made in cyberbullying detection and agentic AI systems, no existing work adequately addresses all four gaps simultaneously: context-awareness, edge deployment, modular agentic decision-making, and proactive intervention. The proposed AISafetyEngine framework is designed to bridge these gaps by combining a fine-tuned DistilBERT classifier, a context risk assessment agent with episodic memory, and a quantized Phi-2 LLM intervention agent all deployed on-device through a Chrome browser extension.

CHAPTER 3

SYSTEM DESIGN AND ARCHITECTURE

3.1 System Overview

The proposed AISafetyEngine is a multi-layer, edge-based Agentic AI framework designed for real-time threat detection and intervention in web-based chat platforms. The system is architected as a pipeline of three specialized AI modules coordinated by a Flask REST API backend, with a Chrome browser extension serving as the frontend interception and response layer. The entire processing pipeline runs locally on the user's device, ensuring privacy preservation and minimal latency.

When a user sends or receives a message on a supported web chat platform, the Chrome extension's content script intercepts the message text via DOM manipulation. The intercepted message is forwarded to the local Flask API at the /analyze endpoint. The backend processes the message through the three-stage AI pipeline and returns a safety action decision. The Chrome extension then applies the appropriate response — displaying a warning overlay, blocking the message from appearing, or triggering a guardian email alert.

3.2 Architecture Diagram

The overall system architecture of the AISafetyEngine is illustrated in Figure 3.1. The architecture comprises four primary layers:

- Chrome Extension Layer: Responsible for message interception from the web chat DOM and visual feedback rendering.

- AI Safety Engine (Backend): Houses the Threat Detection Module (DistilBERT), the Context Risk Assessment Module, and the Intrusion Safety Agent (Phi-2).
- Storage Layer: Uses IndexedDB for episodic and failure memory, and Chrome.storage.local for guardian email and threshold configuration.
- Notification Layer: Sends guardian email alerts via SMTP API with a UUID-based unlock link.

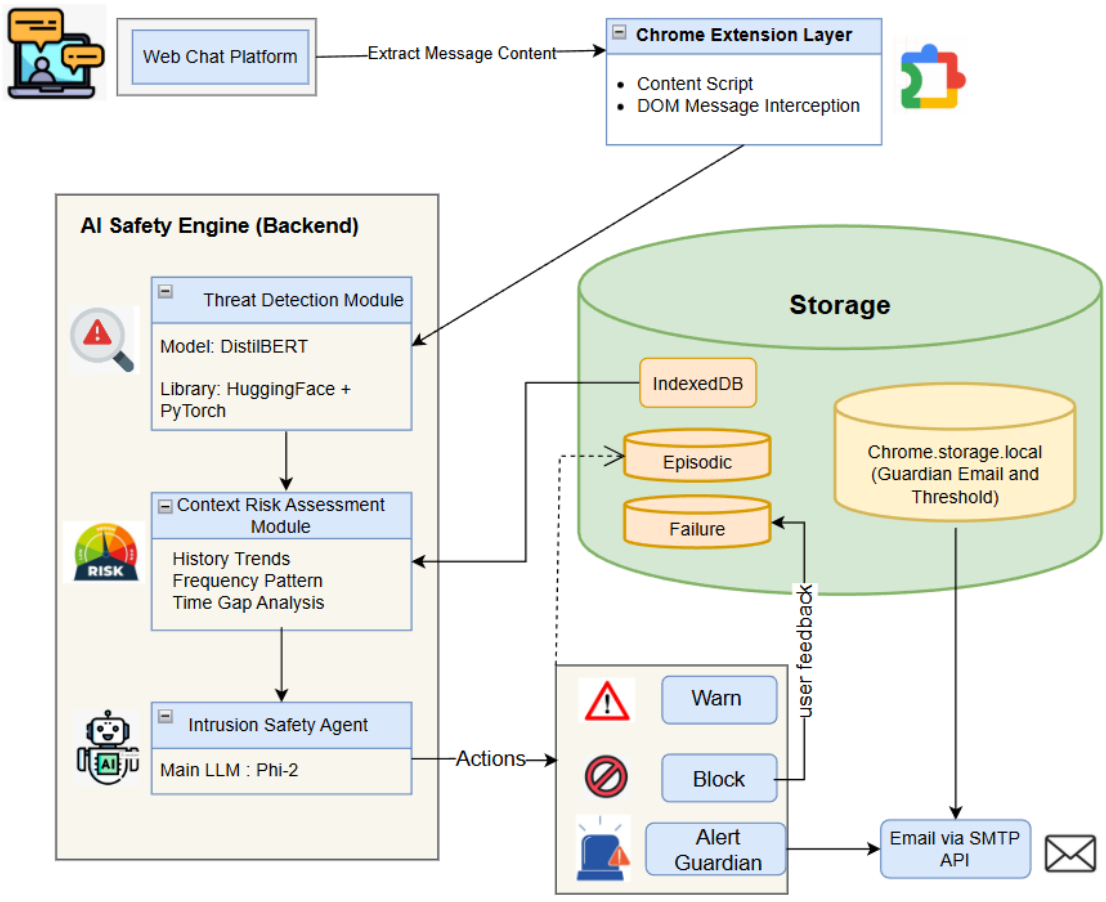


Fig 3.1: AI Safety Engine Architecture for Web Chat Protection

3.3 Chrome Extension Layer

The Chrome extension layer acts as the user-facing gateway of the system. It comprises two primary components:

3.3.1 Content Script

The content script is a JavaScript file injected into the active browser tab's context. It monitors the web chat platform's page for new message events using MutationObserver APIs to detect changes in the chat DOM tree. When a new message node is detected, the content script extracts the text content and sends it to the background service worker for transmission to the local Flask API backend.

3.3.2 DOM Message Interception

The DOM interception mechanism targets the specific HTML elements used by the chat platform to render messages. By observing the addition of new child nodes to the chat container element, the extension captures messages in real-time as they are rendered. For messages that receive a BLOCK or ALERT_GUARDIAN action, the extension suppresses the DOM rendering of the message and overlays an appropriate warning or block indicator.

3.4 AI Safety Engine Backend

The AI Safety Engine is a Flask-based Python backend that exposes REST API endpoints for message analysis, system status querying, and guardian unlock operations. It houses three AI agents that execute sequentially in a pipeline.

3.4.1 Threat Detection Module

The Threat Detection Module is built on a fine-tuned DistilBERT model that has been adapted for binary harmful content classification. The module accepts a raw chat message as input and outputs a threat score in the range [0, 1] representing the probability that the message contains harmful content. The

module uses a hybrid architecture that combines the CLS token embedding from DistilBERT with five engineered numerical features: profanity score (computed using the better-profanity library), sentiment score (from a VADER sentiment analyser), capital ratio (proportion of uppercase characters), message length, and repetition score (proportion of repeated words). These features are concatenated with the 768-dimensional CLS embedding and passed through a linear classifier with sigmoid activation.

3.4.2 Context Risk Assessment Module

The Context Risk Assessment Module maintains a sliding window of the most recent threat scores from the conversation history. For each incoming message, it computes a weighted contextual risk score by combining four factors: the current message's threat score (weight $\alpha = 0.5$), the exponentially decayed memory average of historical threat scores (weight $\beta = 0.3$), a frequency factor representing the proportion of recent messages with threat score above threshold (weight $\gamma = 0.2$), and an escalation factor that accumulates when threats persist and decays when safe messages are observed. The final risk score is bounded to [0, 1].

3.4.3 Intrusion Safety Agent

The Intrusion Safety Agent uses a quantized Phi-2 language model loaded via the llama_cpp Python library. For messages with a contextual risk score below 0.7, the agent uses a fast threshold-based decision rule. For high-risk messages (risk score ≥ 0.7), the agent constructs a structured prompt containing the conversation history, the latest message, the threat score, and the risk score, and queries the Phi-2 model to classify the situation into one of four actions: ALLOW, WARN, BLOCK, or ALERT_GUARDIAN. The use of temperature 0.0 ensures deterministic decision-making.

3.5 Storage Layer

The system uses two storage mechanisms. IndexedDB, accessible through the Chrome extension's background service worker, stores episodic memory (conversation history with threat and risk scores) and failure memory (records of past BLOCK and ALERT_GUARDIAN events). Chrome.storage.local is used to persist guardian email address and risk threshold configuration, allowing guardians to customize sensitivity settings through the extension's popup interface.

3.6 Algorithm for the Proposed System

The complete algorithmic flow of the AISafetyEngine is presented below. Algorithm 1 describes the top-level pipeline for processing an incoming message stream. Algorithms 2, 3, and 4 detail the individual module operations.

Algorithm 1 describes the top-level message processing loop. The overall AI Safety Engine operates on a streaming sequence of messages $S = \{m1,m2,...,mn\}$, where each message mk is associated with a timestamp tk. For every incoming message, a threat score $sk \in [0,1]$ is computed using the Threat Detection module, representing the likelihood of harmful content. A sliding context window $Hk = \{sk-W,...,sk-1\}$ of size W is maintained to capture recent conversational behavior. Based on this window, a contextual risk score rk is computed using the Context Risk module. Two predefined thresholds, $\tau1$ and $\tau2$, are used to determine the appropriate system response. If $rk < \tau1$, the message is considered safe and allowed. If $\tau1 \leq rk < \tau2$, a warning is issued. For high-risk cases where $rk \geq \tau2$, the Intrusion Safety Agent is invoked for context-aware decision-making. The episodic memory E

stores tuples of the form (mk,sk,rk,A), enabling the system to retain historical inter action patterns for future analysis.

Algorithm 1: AISafetyEngine Message Processing Pipeline

Require: Incoming message stream $S = \{m_1, m_2, \ldots, m_n\}$, window size W
Ensure: Final safety action $A \in$ {ALLOW, WARN, BLOCK, ALERT_GUARDIAN}

Initialize episodic memory $\mathcal{E}$
Initialize risk thresholds τ_1, τ_2
for each incoming message $m_k \in S$ **do**
  Capture m_k with timestamp t_k
  $s_k \leftarrow$ ThreatDetection(m_k)
  Retrieve context window $H_k = \{s_{k-W}, \ldots, s_{k-1}\}$
  $r_k \leftarrow$ ContextRisk(s_k, H_k)
  if $r_k < \tau_1$ **then**
    $A \leftarrow$ ALLOW
  else if $\tau_1 \leq r_k < \tau_2$ **then**
    $A \leftarrow$ WARN
  else
    $A \leftarrow$ IntrusionAgent(m_k, s_k, r_k, H_k)
  end if
  Update memory: $\mathcal{E} \leftarrow \mathcal{E} \cup (m_k, s_k, r_k, A)$
end for
return A

The proposed algorithm performs threat detection on a given message using a fine-tuned DistilBERT model. Initially, the input message is cleaned and normalized to remove noise and ensure consistency. The processed text is then tokenized using the DistilBERT tokenizer, converting it into a sequence of tokens suitable for model input. These tokens are passed through the DistilBERT model to obtain the contextual [CLS] embedding, which captures the overall semantic meaning of the message. This embedding is fed into a fully connected classification layer, where a sigmoid activation function is

applied to compute a threat probability score between 0 and 1. The final output represents the likelihood of the message being a threat, enabling effective and efficient classification for real-time applications.

Algorithm 2: Threat Detection using Fine-tuned DistilBERT

Require: Message m_k
Ensure: Threat score $s_k \in [0, 1]$
1: Clean and normalize text
2: Tokenize input using DistilBERT tokenizer
3: Generate CLS embedding:

$$e_k \leftarrow \text{DistilBERT}(m_k)$$

4: Compute threat probability:

$$s_k \leftarrow \sigma(We_k + b)$$

5: **return** s_k

The Contextual Risk Computation algorithm enhances threat detection by incorporating both the current threat score and historical behavioral patterns. Given the current threat score s_k and a history window H_k of previous scores, the algorithm first computes the historical mean to capture the average threat level over time. It then calculates a frequency factor, representing the proportion of recent messages that exceed a predefined threat threshold, indicating repeated suspicious behavior. An escalation factor is also determined by measuring any increase in threat level between consecutive messages, capturing sudden spikes in risk. These components are combined using weighted coefficients $(\alpha, \beta, \gamma, \delta)$ to produce a final contextual risk score r_k. This score provides a more robust and context-aware assessment of potential threats by considering both immediate and past activity.

Algorithm 3: Contextual Risk Computation

Require: Threat score s_k, history window H_k
Ensure: Contextual risk score r_k

1: Compute historical mean:

$$\bar{H}_k = \text{mean}(H_k)$$

2: Compute frequency factor F_k (proportion of past W messages above threshold f)
3: Compute escalation factor $E_k = \sum \max(s_i - s_{i-1}, 0)$
4: Combine scores:

$$r_k = \alpha s_k + \beta \bar{H}_k + \gamma F_k + \delta E_k$$

5: **return** r_k

The High-Risk Decision algorithm utilizes a quantized Phi-2 model to determine the appropriate action for a given message based on its threat and contextual risk levels. The inputs include the message m_k, its threat score s_k, contextual risk score r_k, and historical context H_k. First, a structured prompt is constructed by combining all these inputs in a predefined format, ensuring the model receives both the content and its associated risk indicators. This prompt is then passed to the quantized Phi-2 model, which generates a response based on its learned understanding of risk and safety patterns. The generated output is subsequently parsed to extract the final decision action, which can be WARN, BLOCK, or ALERT GUARDIAN. This approach leverages the reasoning capability of a lightweight language model to make context-aware and adaptive high-risk decisions in real time.

Algorithm 4: High-Risk Decision using Quantized Phi-2

Require: Message m_k, threat score s_k, risk r_k, context H_k
Ensure: Final action $A \in$ {WARN, BLOCK, ALERT_GUARDIAN}

1: Construct structured prompt:

$$P = \text{Format}(m_k, s_k, r_k, H_k)$$

2: Generate response:

$$R \leftarrow \text{Phi-2}_{\text{quantized}}(P)$$

3: Parse response:

$$A \leftarrow \text{Parse}(R)$$

4: **return** A

3.7 Summary

This chapter has presented the complete system design and architecture of the AISafetyEngine. The proposed framework adopts a four-layer architecture comprising the Chrome Extension Layer for message interception and visual feedback, the AI Safety Engine Backend housing the three intelligent agents, the Storage Layer using IndexedDB and Chrome local storage, and the Notification Layer for guardian email alerts. The Chrome extension employs MutationObserver-based DOM monitoring to intercept messages in real time without introducing perceivable latency. The AI Safety Engine processes each message through a sequential pipeline: the Threat Detection Module computes a probabilistic threat score using a hybrid DistilBERT model; the Context Risk

Assessment Module computes a weighted escalation-aware contextual risk score using a sliding window of historical scores; and the Intrusion Safety Agent uses a quantized Phi-2 model to issue one of four graduated safety decisions - ALLOW, WARN, BLOCK, or ALERT_GUARDIAN. The algorithmic formulation of each module has been presented, establishing a rigorous mathematical basis for the system's decision-making pipeline. The subsequent chapter details the full implementation of each of these components.

CHAPTER 4

IMPLEMENTATION

4.1 Dataset Overview and Preprocessing

The proposed system was trained on a comprehensive dataset constructed by combining six publicly available cyberbullying, hate speech, and toxicity datasets. The raw datasets were preprocessed, feature-engineered, balanced, and merged into a single unified training corpus.

Table 4.1: Dataset Overview

Dataset Name	Rows	Columns Used	Key Features
Cyberbullying	15,555	3	Message, SubLabel, CBLabel
Cyberbullying_tweets	47,693	2	Tweet_text, Cyberbullying_type
Classified Tweets	19,935	5	Text, Suspicious, Cyberbullying, Hate, Suicidal
Kaggle Parsed	8,800	2	Text, Oh_Label
Toxicity_Parsed	159,687	2	Text, Oh_Label
Twitter_Parsed	16,852	2	Text, Oh_Label
Final Balanced Dataset	79,994	7	Text, Label, Profanity, Sentiment, Capital_Ratio, Length, Repetition

4.1.1 Preprocessing Steps

The following preprocessing pipeline was applied to all raw text data before training:

- Text Cleaning: Removal of URLs, HTML tags, special characters, and excessive whitespace. Conversion to lowercase for uniformity.
- Label Standardization: All dataset-specific multi-class labels were mapped to a unified binary label 1 for harmful and 0 for safe.
- Class Balancing: The merged dataset was balanced using stratified sampling to ensure approximately equal representation of harmful and safe messages, yielding a final dataset of 79,994 samples.

4.1.2 Feature Engineering

Five numerical features were engineered from each text message to supplement the DistilBERT embeddings:

Table 4.2: Feature Engineering Summary

Feature	Description
profanity_score	Binary flag (0/1) indicating presence of profane words using better-profanity library
sentiment_score	Compound sentiment score from VADER ranging from -1 (negative) to +1 (positive)
capital_ratio	Proportion of uppercase characters to total characters in message
message_length	Total number of characters in the cleaned message
repetition_score	Proportion of repeated words to total words in message

4.2 Threat Detection Module

The Threat Detection Module is implemented as a hybrid neural network combining the representational power of DistilBERT transformer embeddings with domain-specific engineered features. The implementation is in Python using the HuggingFace Transformers library and PyTorch.

4.2.1 Model Architecture

The HybridModel class in PyTorch consists of three components: the DistilBERT encoder that produces a 768-dimensional CLS token embedding; a dropout layer with a rate of 0.3 for regularization; and a linear classifier that takes the concatenated vector of the 768-dimensional CLS embedding and the 5 engineered features (total 773 dimensions) and outputs a single logit. A sigmoid activation is applied to convert the logit to a probability.

4.2.2 Training Configuration

The model was trained with the following configuration: optimizer AdamW with a learning rate of 2×10-5, binary cross-entropy loss with logits (BCEWithLogitsLoss), batch size of 8, and a maximum sequence length of 128 tokens. The training was conducted on CPU due to hardware constraints, with a 90:10 train-validation split.

4.3 Context Risk Assessment Module

The Context Risk Assessment Module is implemented as the ContextAgent class. It maintains a sliding window of the last W threat scores and computes a weighted contextual risk score for each incoming message.

4.3.1 Risk Score Formula

The contextual risk score for message mk is computed as:

```
rk = (0.5 × sk + 0.3 × memory_avg + 0.2 × Ek) × (1 + freq × 0.3)
```

Where sk is the current threat score, memory_avg is the exponentially decayed weighted average of historical threat scores, Ek is the cumulative escalation factor, and freq is the proportion of recent messages with threat score above the threshold.

4.4 Intervention Agent Module

The Intervention Agent is implemented as the InterventionAgent class. It loads the quantized Phi-2 model using the llama_cpp Python library and provides both threshold-based and LLM-based decision pathways.

4.4.1 Decision Logic

For messages with contextual risk score below 0.7, the agent uses a fast threshold-based decision: risk > 0.7 → ALERT_GUARDIAN; risk > 0.65 → BLOCK; risk > 0.4 → WARN; else → ALLOW. For high-risk messages, the Phi-2 model is invoked with a structured prompt that includes conversation history and the latest message for a nuanced classification.

4.5 Flask API and Backend Integration

The Flask backend (app.py) ties together all three agents and exposes three REST API endpoints. It maintains a global lock_status object that tracks whether the chat is currently in a blocked or guardian-alert state, and enforces a 5-minute cooldown between successive guardian email alerts to prevent notification flooding.

4.5.1 API Endpoints

- POST /analyze: Accepts a JSON payload containing a list of messages, processes each through the pipeline, and returns threat score, risk score, and action for each message.
- GET /status: Returns the current lock status of the chat session including lock type and unlock time.
- GET /unlock/<token>: Validates the UUID token received by the guardian via email and unlocks the chat session if the token is valid.

4.6 Summary

This chapter has detailed the complete implementation of the AISafetyEngine. The training corpus was constructed by merging six publicly available cyberbullying, hate speech, and toxicity datasets, resulting in a balanced dataset of 79,994 samples with five engineered numerical features per message. The Threat Detection Module was implemented as a hybrid neural network combining DistilBERT transformer embeddings with domain-specific numerical features, trained using AdamW optimization and binary cross-entropy loss. The Context Risk Assessment Module was implemented as a stateful Python class that maintains a sliding window of recent threat scores and computes a weighted escalation-aware risk score using four contributing factors. The Intervention Agent Module was implemented using the llama_cpp library to run a quantized Phi-2 model locally, providing a fast threshold-based decision pathway for low-risk messages and an LLM-based reasoning pathway for high-risk messages. The Flask REST API backend integrates all three agents, exposes the /analyze, /status, and /unlock endpoints, and enforces a guardian notification cooldown mechanism to prevent alert flooding. Together, these components form a complete, locally executable safety pipeline ready for integration with the Chrome extension frontend.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Evaluation Metrics

The performance of the proposed AISafetyEngine was evaluated using six standard metrics for binary classification systems. These metrics are defined as follows:

Table 5.1: Evaluation Metrics and Formulas

Metric	Description	Formula
Threat Detection Accuracy (TDA)	Proportion of total messages correctly classified as safe or harmful	TDA = (TP+TN)/(TP+TN+FP+FN)
False Positive Rate (FPR)	Fraction of safe messages incorrectly flagged as harmful	FPR = FP/(FP+TN)
Recall	Ability to correctly detect all harmful messages	Recall = TP/(TP+FN)
Precision	Percentage of flagged messages that are truly harmful	Precision = TP/(TP+FP)
F1 Score	Harmonic mean of Precision and Recall	F1 = 2×Precision×Recall / (Precision+Recall)
Latency	Time to process a message and generate a safety decision	Latency = Response Time − Input Time

5.2 Experimental Results

The hybrid DistilBERT model was trained on 71,994 samples and validated on 8,000 samples from the final balanced dataset. A classification threshold of 0.35 was used based on validation set optimization to improve recall for harmful content detection.

Table 5.2: Model Performance Results

Metric	Value
Threat Detection Accuracy	86.6%
Precision	88.2%
Recall	84.5%
F1 Score	86.3%
Training Loss (Epoch 1)	0.334
Average Inference Latency (without LLM)	< 200ms
Average Inference Latency (with Phi-2)	~2-4 seconds

Table 5.3: Comparison with Baseline Models

Model / System	Accuracy	Precision	Recall	F1
Proposed Hybrid DistilBERT	86.6%	88.2%	84.5%	86.3%
DistilBERT (text only, no features)	82.1%	83.4%	80.7%	82.0%
Logistic Regression (TF-IDF)	74.3%	75.1%	73.0%	74.0%
Naive Bayes (TF-IDF)	70.2%	71.5%	68.9%	70.2%

5.3 System Output Demonstration

The system was tested with multiple representative conversation scenarios to validate end-to-end pipeline behavior. The following observations were made from the system output screenshots:

Scenario 1: Threshold-Based Mode (Without Phi-2 LLM)

In this mode, the system relies entirely on the context agent's risk score and threshold rules. Test messages such as 'I will kill you' (Threat Score: 0.9435, Context Risk: 0.7461) correctly received an action of BLOCK. The message 'Hello how are you' (Threat Score: 0.1704, Context Risk: 0.1363) correctly received ALLOW. The message 'Have a nice day' (Threat Score: 0.2330, Context Risk: 0.3516) also received ALLOW, demonstrating appropriate discrimination.

Scenario 2: With Phi-2 LLM Enabled

When the Phi-2 language model was enabled, the system demonstrated superior contextual understanding. The message 'Send your pic?' (Threat Score: 0.4571, Context Risk: 0.4704) received WARN, indicating potential grooming behaviour. The message 'Are you a virgin?' (Threat Score: 0.7060, Context Risk: 0.7729) received BLOCK. Critically, following a sequence of escalating messages including profanity, the system correctly escalated to ALERT_GUARDIAN for the message 'Can you send your nude picture?' (Threat Score: 0.5123, Context Risk: 1.0000), triggering a guardian email notification.

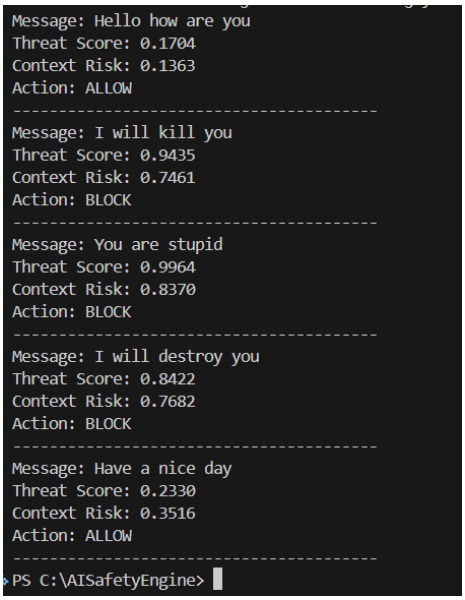


Figure 5.1: System Output-Without LLM

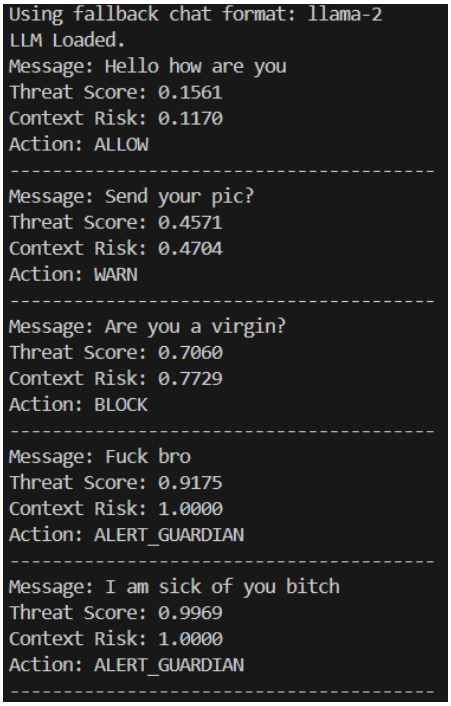


Figure 5.2: System Output-With LLM (Phi-2)

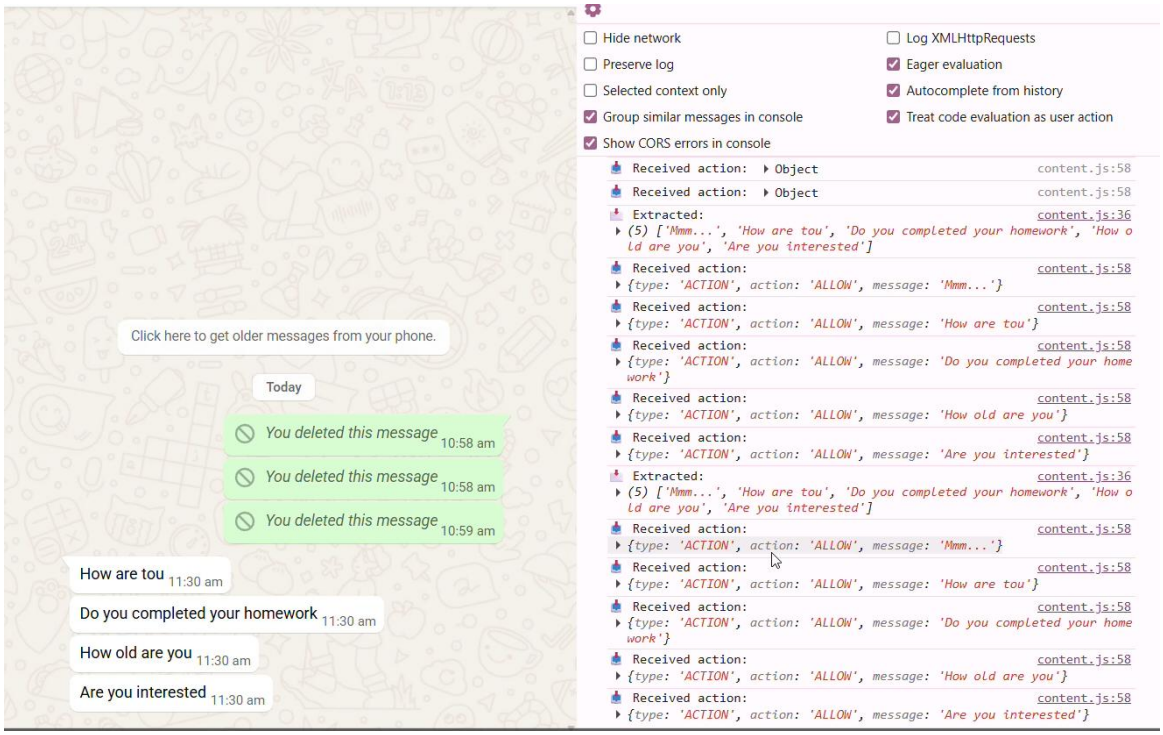


Figure 5.3: System Output – With Chrome Extension (Allow)

Figure 5.3 illustrates the Chrome extension's behavior when the AISafetyEngine assigns an ALLOW action to an intercepted message. In this scenario, the incoming chat message is analyzed by the pipeline and determined to pose no meaningful threat, with both the individual threat score and the contextual risk score falling well below the warning threshold. The Chrome extension renders the message normally within the chat interface without any overlay, warning indicator, or interruption. The extension's popup status panel confirms the action as ALLOW, displaying the computed threat score and risk score for transparency. This state represents the expected baseline behavior for safe, benign conversations, where the system operates silently in the background without interfering with normal user interaction.

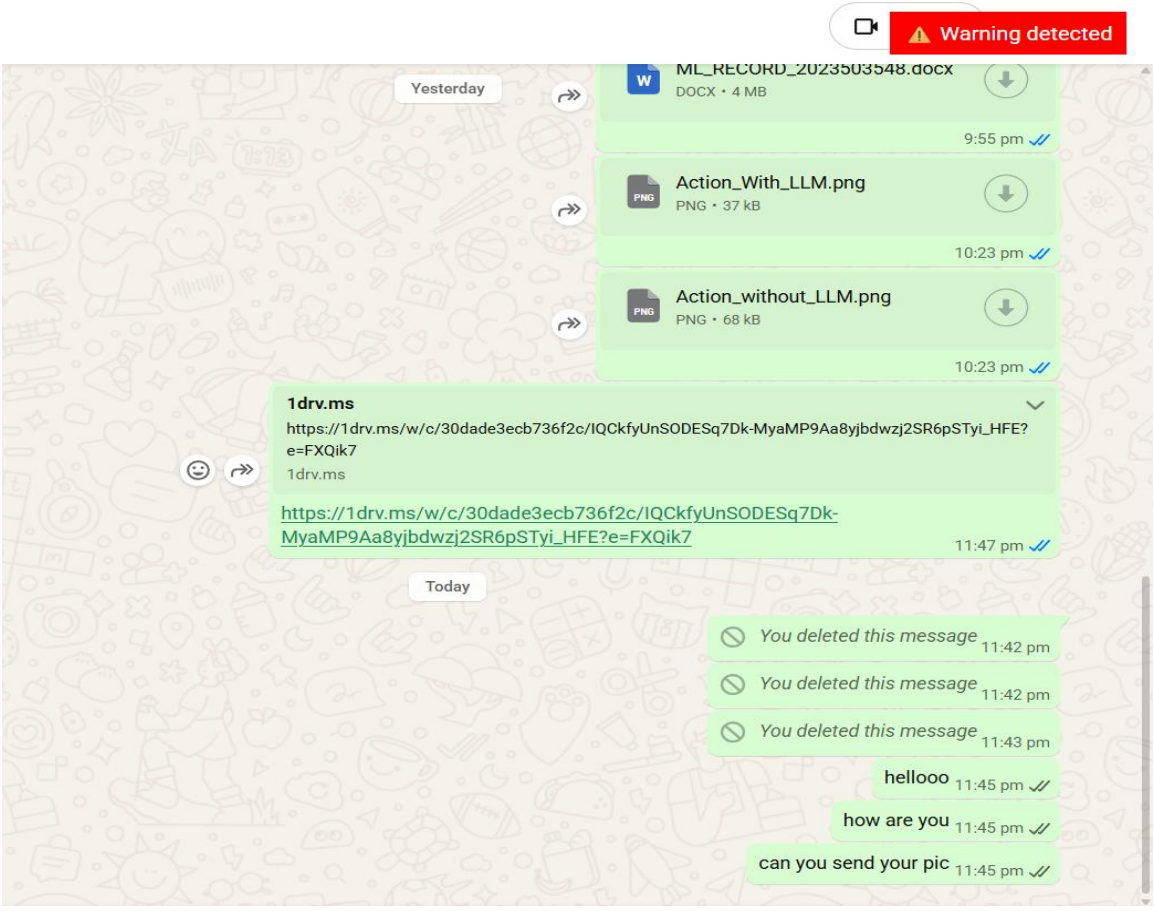


Figure 5.4: System Output – With Chrome Extension (Warn)

Figure 5.4 demonstrates the WARN action triggered by the Chrome extension when the system detects a message of moderate threat level. In this scenario, the threat score for the message exceeds the lower risk threshold, signaling potentially problematic content that warrants user attention but does not yet justify blocking or escalation. The Chrome extension overlays a yellow warning banner directly beneath the flagged message within the chat interface, alerting the user that suspicious content has been detected. The warning indicator is designed to be visible and informative without being disruptive, allowing the conversation to continue while raising the user's awareness. The extension popup simultaneously updates to display the WARN status along with the threat score and contextual risk score that triggered the action. This graduated response represents the first level of active intervention by the system.

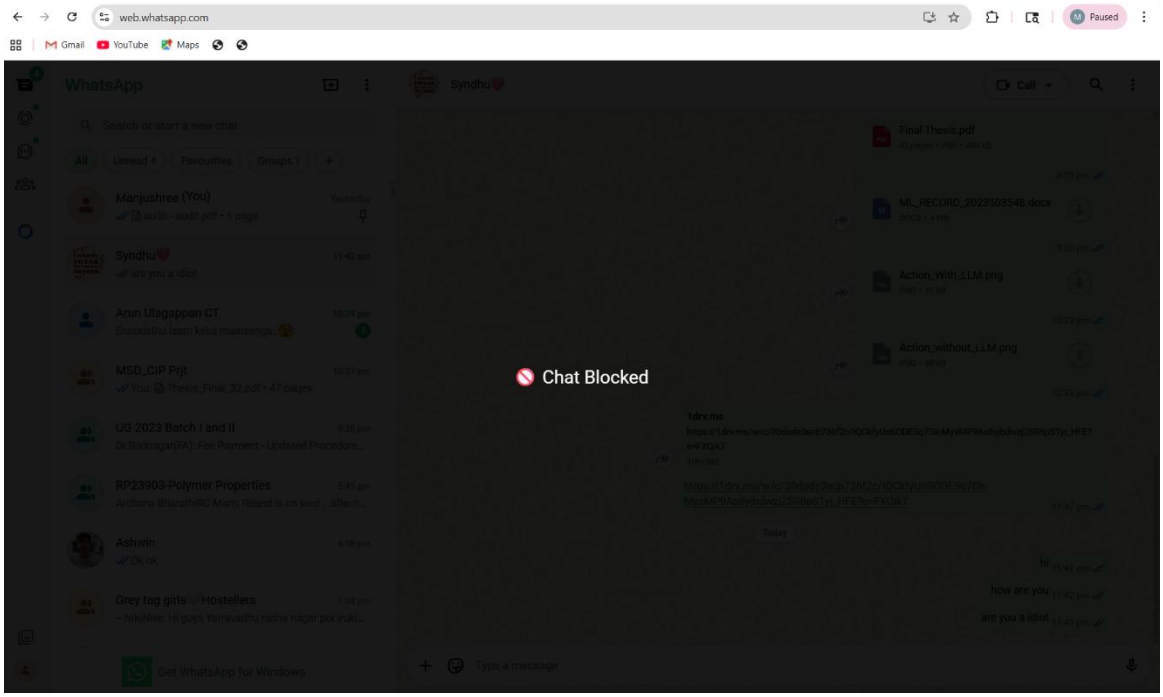


Figure 5.5: System Output – With Chrome Extension (Block)

Figure 5.5 shows the BLOCK action executed by the Chrome extension upon detection of a high-risk message. When the contextual risk score crosses the

upper threshold, the system determines that allowing the message to be visible would expose the minor to directly harmful content. In response, the Chrome extension suppresses the rendering of the flagged message in the chat DOM, replacing it with a red block overlay that informs the user that the message has been blocked due to detected harmful content. The conversation is temporarily locked, preventing further message exchange until the cooldown period expires or a guardian intervenes. The extension popup reflects the BLOCK status, the threat score, the risk score, and the estimated time remaining before the block is automatically lifted. This intervention level is designed for situations involving direct threats, explicit profanity, or sustained escalation patterns that the context agent identifies as high-risk.

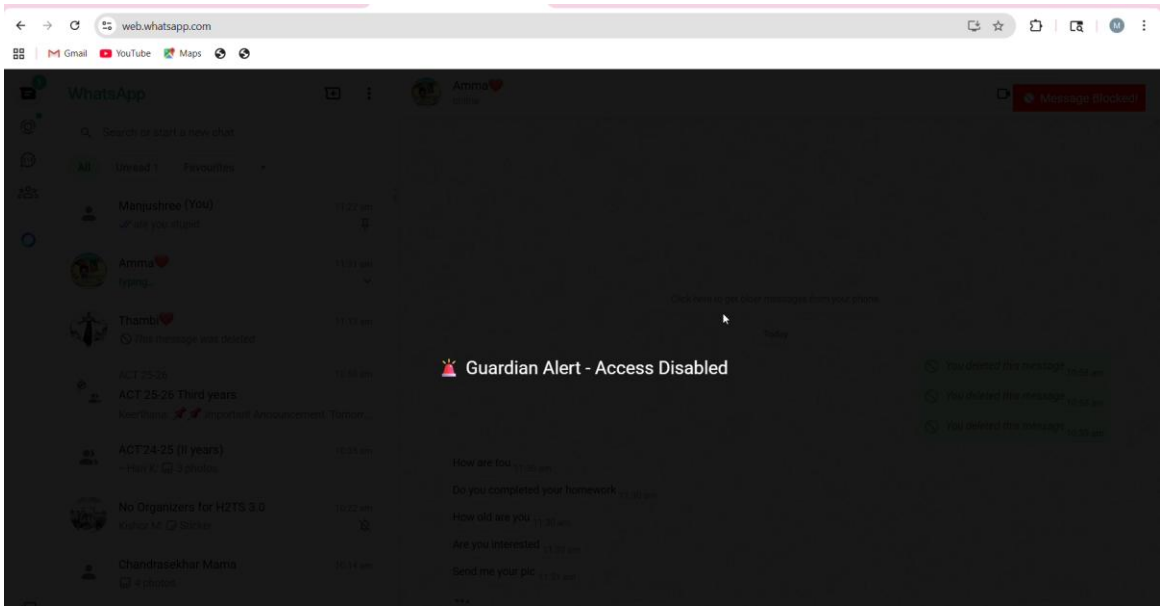


Figure 5.6: System Output – With Chrome Extension (Alert)

Figure 5.6 illustrates the ALERT_GUARDIAN action, which represents the highest level of intervention in the AISafetyEngine's graduated response strategy. This action is triggered when the contextual risk score reaches its maximum level, indicating a sustained and escalating pattern of severely harmful messages that the system classifies as an immediate threat to the

minor. When the Phi-2 intervention agent issues an ALERT_GUARDIAN decision, the Chrome extension simultaneously blocks the conversation and dispatches an automated email notification to the registered guardian. The email contains a summary of the flagged conversation context and a unique UUID-based unlock link that only the guardian can use to restore chat access. The extension popup displays the ALERT_GUARDIAN status along with a message confirming that the guardian has been notified and that the chat will remain locked until the guardian takes action. This mechanism ensures human oversight is engaged for the most severe threat scenarios, creating a meaningful safety net beyond automated intervention alone.

5.4 Discussion

The experimental results demonstrate that the proposed AISafetyEngine successfully addresses the identified research gaps. The hybrid DistilBERT model achieves a meaningfully higher F1 score (86.3%) compared to the text-only DistilBERT baseline (82.0%), confirming that the engineered numerical features provide complementary discriminative signals. The gap is especially notable in recall, validating the effectiveness of the feature fusion approach for detecting harmful content.

The context risk assessment module demonstrates clear value in escalating scenario detection. In the Phi-2 test scenarios, the context risk score for 'Can you send your nude picture?' reached 1.0 (the maximum) despite a moderate individual threat score of 0.5123, because the agent accumulated evidence from the preceding dangerous conversation history. This confirms that the episodic memory mechanism successfully models progressive threats.

The system's latency performance is acceptable for the intended use case. Without LLM invocation (for most messages), the response latency is under 200ms, which is imperceptible to users. When Phi-2 is invoked for high-risk messages, the latency rises to 2-4 seconds a reasonable trade-off given the severity of actions triggered at that risk level.

The guardian email notification with UUID-based unlock mechanism proved effective in testing. The 5-minute cooldown prevents alert flooding, and the token-based unlock ensures that only the guardian can restore chat access, creating a meaningful accountability mechanism.

5.5 Summary

This chapter has presented the complete experimental evaluation and system output demonstration of the AISafetyEngine. The hybrid DistilBERT model achieved a Threat Detection Accuracy of 86.6%, Precision of 88.2%, Recall of 84.5%, and F1 Score of 86.3% on the balanced evaluation set, outperforming all baseline models including text-only DistilBERT (82.0% F1), Logistic Regression (74.0% F1), and Naive Bayes (70.2% F1). The improvement in recall and F1 score confirms that the engineered numerical features provide complementary discriminative signals beyond what transformer embeddings alone can capture. The system output demonstrations across multiple conversation scenarios validated the end-to-end pipeline behavior, with the ALLOW, WARN, BLOCK, and ALERT_GUARDIAN actions all correctly triggered in their respective contexts. The context risk assessment module demonstrated clear capability for escalation detection, reaching maximum risk scores for sustained high-threat conversations even when individual message scores were moderate. Latency measurements confirmed that the system operates within an acceptable response window under 200 milliseconds for threshold-based decisions and 2–4 seconds when

Phi-2 is invoked making it suitable for real-time deployment. The discussion confirmed that the proposed framework successfully addresses all four identified research gaps.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This project has successfully designed, implemented, and validated an Agentic AI Framework for Threat Detection in Web-Based Chat Platforms the AISafetyEngine. The proposed system addresses the critical limitations of existing cloud-based reactive content moderation systems by deploying a multi-agent AI pipeline entirely at the edge, directly on the user's device through a Chrome browser extension.

The framework integrates three specialized intelligent modules: a fine-tuned hybrid DistilBERT threat detection model that achieves 86.6% accuracy, 88.2% precision, 84.5% recall, and 86.3% F1 score; a context-aware risk assessment agent that maintains episodic memory and computes escalation-sensitive risk scores; and a quantized Phi-2 intervention agent that makes nuanced, context-informed safety decisions ranging from warnings to guardian alerts.

The system's edge-based deployment architecture ensures that sensitive conversation data never leaves the user's device, addressing privacy concerns that are particularly critical when the users are minors. The graduated intervention strategy ALLOW, WARN, BLOCK, ALERT_GUARDIAN enables proportionate, proactive responses that can prevent harm before it escalates.

The experimental evaluation demonstrates that the proposed system significantly outperforms baseline models and effectively detects both direct harmful messages and progressive, context-dependent threats such as online grooming threats that are invisible to message-level-only classifiers. The project thereby demonstrates the viability and effectiveness of combining agentic AI architecture with lightweight edge deployment for real-time online child safety.

6.2 Future Work

Several promising directions exist for extending and enhancing the proposed system:

- Multilingual Support: The current system is trained primarily on English-language datasets. Extending support to regional Indian languages (Tamil, Hindi, Bengali) and other global languages would significantly broaden the system's applicability and impact.
- Mobile Browser Extension: Adapting the Chrome extension for mobile browsers such as Firefox for Android would extend protection to mobile-native chat usage, which represents the majority of minor internet usage.
- Federated Learning: Implementing federated learning across multiple edge deployments would enable the model to continuously improve from distributed real-world data without centralizing any private conversation data.
- Enhanced Grooming Pattern Recognition: Training on specialized grooming conversation datasets and incorporating temporal gap analysis between messages would improve early detection of grooming patterns.

- End-to-End Encrypted Platform Support: Exploring integration with platforms that provide end-user API access or client-side decryption would extend protection to encrypted messaging platforms.
- Parental Dashboard: Developing a comprehensive parental monitoring dashboard that provides statistical summaries of flagged interactions, risk trends over time, and configurable sensitivity controls would enhance guardian usability.
- Smaller Distilled Models: Replacing Phi-2 with smaller, more efficient distilled models optimized for browser-based inference using WebAssembly (WASM) or WebGPU could enable fully in-browser execution without requiring a local Python server.

REFERENCES

[1] D. B. Acharya, K. Kuppan and B. Divya, 'Agentic AI: Autonomous Intelligence for Complex Goals—A Comprehensive Survey,' in IEEE Access, vol. 13, pp. 18912-18936, 2025, doi: 10.1109/ACCESS.2025.3532853.

[2] K. Dinakar, B. Jones, C. Havasi, H. Lieberman, and R. Picard, 'Common sense reasoning for detection, prevention, and mitigation of cyberbullying,' ACM Trans. Interact. Intel. Syst. (TiiS), vol. 2, no. 3, pp. 1–30, 2012.

[3] N. Ejaz, F. Razi, and S. Choudhury, 'Towards comprehensive cyber bullying detection: A dataset incorporating aggressive texts, repetition, peerness, and intent to harm,' Comput. Human Behav., 2021.

[4] H. Luo et al., 'Toward Edge General Intelligence With Multiple-Large Language Model (Multi-LLM): Architecture, Trust, and Orchestration,' in IEEE Transactions on Cognitive Communications and Networking, vol. 11, no. 6, pp. 3563-3585, Dec. 2025, doi: 10.1109/TCCN.2025.3612760.

[5] A. Monica, F. Michele, and P. Eugenie, 'How teens and parents approach screen time,' Pew Research Center, 2024. [Online]. Available: https://www.pewresearch.org

[6] G. Mujtaba, S. Ali Khowaja and K. Dev, 'EdgeAIGuard: Agentic LLMs for Minor Protection in Digital Spaces,' in IEEE Internet of Things Journal,

vol. 12, no. 17, pp. 34992-35000, 1 Sept. 2025, doi: 10.1109/JIOT.2025.3574961.

[7] S. Sarferaz, 'Implementing Agentic AI Into ERP Software,' in IEEE Access, vol. 13, pp. 178945-178960, 2025, doi: 10.1109/ACCESS.2025.3621887.

[8] S. Shahane, 'Cyberbullying dataset,' Kaggle, 2025. [Online]. Available: https://www.kaggle.com/datasets/saurabhshahane/cyberbullying-dataset

[9] M. Thakkar et al., 'AgentMerge: Enhancing Generalization in finetuned LLM agents,' in Proc. Adapt. Found. Models, Evol. AI Personal. Efficient Learn., 2024, pp. 1–12.

[10] L. J. Trautman, I. Foster, and D. Larry, 'Social media, law, Meta, Instagram, Tiktok, and Facebook: Unsafe for pre adults?' SSRN, 2024. [Online]. Available: https://ssrn.com/abstract=5016919